claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-aedaf9ed6be321b62.jsonl`
- SHA-256: `f6f2267ed276b2f46768326fe938cc8e350043fb0a3bc7a0018bf07586737b4a`
- Source modified: `2026-07-06T09:10:06+00:00`
- Imported at: `2026-07-08T16:00:05+00:00`
- project: `wf_3512746c-54e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T09:09:16.744Z)

```
Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-
selective-scan -> main, one commit db87cc0).
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels,
/watchlist commands +
an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY:
command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py
(set_source_watched, list_account_sources), telethon_client.py (the gate),
tests/test_command_service.py (10 tests).
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must
conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):
- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans
nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be
explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear
in /channels for opt-in;
  /channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:
- READ actual code (Read/Grep), cite file:line. You MAY run repros with
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema
  on aiosqlite ":memory:"; seed via db.upsert_account + db.upsert_source; drive
command_service.CommandService().handle()).
  NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g.
'-1001234567890'); upsert_channel
  creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover'
function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest
492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value
finding. No praise.
  Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-
contract violation users hit),
  medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND
docs/adr/0009-selective-channel-scanning.md,
trace real control flow, run a repro with
```

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff
and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur /
pre-existing. PLAUSIBLE if unsettled. Corrected severity.
FINDING (tests_ux): Ingestion gate has zero test coverage (inlined in a # pragma: no cover handler)
severity high | src/tg_video_filter/telethon_client.py:184
summary: The core Track E gating logic lives entirely inside on_new_message, which is marked '# pragma: no cover' (telethon_client.py:167), so the 97.10% coverage figure excludes it and no test exercises skip-unwatched / pass-watched / new-channel behavior.
scenario: A grep of tests/ for on_new_message, make_new_message_handler, 'watched gate', or 'not src.watched' returns zero matches. Because the gate is a few lines embedded in the excluded handler rather than a pure, callable helper, the auto-watch defect in finding #1 shipped undetected -- a single test asserting 'a new (unseen) channel's first video is NOT delivered' would have failed and caught it. The gating behavior (the entire point of this PR) can regress freely with the suite still green.
evidence: telethon_client.py:167 `async def on_new_message(...) -> None:  # pragma: no cover`; the watched gate is at lines 184-191 inside it. tests/ has no reference to the gate. The new tests in test_command_service.py exercise the /watch etc. commands and db persistence, but nothing drives the ingestion decision.
rec: Extract the gate into a pure, testable helper, e.g. `async def should_ingest(conn, account_id, channel_id) -> bool` (or a synchronous predicate over a fetched Source), that returns False for both unwatched-existing and unseen-new sources. Add tests: (a) watched source -> True, (b) existing unwatched source -> False, (c) previously-unseen channel -> False. This both fixes coverage and locks in the correct opt-in semantics from finding #1.

2. assistant (2026-07-06T09:09:19.538Z)

I'll verify this finding by reading the actual code and checking whether the gate has test coverage.

3. user (2026-07-06T09:09:20.397Z)

```
150         engine_holder: dict[str, FilterEngine],
151         client: TelegramClient,
152         delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
153         delivery_target: str = "me",
154     ) -> _NewMessageCallback:
155         """Return an async callback for ``events.NewMessage``.
156 
157         The callback:
158           1. extracts VideoMeta + document_id (skips if not a video),
159           2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
160              plus the existing (channel, message) instance dedup,
161           3. runs the filter engine (read from ``engine_holder`` so config
162              changes via ``/set`` take effect immediately),
163           4. inserts a VideoRecord row,
164           5. delivers a match to ``delivery_target`` (see ADR 0005).
165         """
166 
167         async def on_new_message(event: NewMessage.Event) -> None:  # pragma: no cover
168             message: Message = event.message  # type: ignore[assignment]
169             if message is None:
170                 return  # Telethon edge case: event has no message (shouldn't happen)
171             # Only process genuine channel/group posts and private messages we
172             # receive ? outgoing messages (sent by our own account) are ignored.
173             if getattr(message, "out", False):
174                 return
175 
176             meta = extract_video_meta(message)
177             if meta is None:
178                 return  # not a video
179 
180             channel_id = _peer_id(message.peer_id)
```

```
            message_id = message.id
            document_id = _extract_document_id(message)

            # --- watched gate (ADR 0009, Track E) -------------------------------
            # Skip ingestion for unwatched sources. The channel must already be in
            # sources (lazy-created by the /channels enumeration or the legacy
            # upsert_channel below); if absent, we let the legacy path upsert it
            # with watched=True for backwards-compatibility.
            src = await db.get_source_by_provider_id(conn, user.id, "telegram",
str(channel_id))
            if src is not None and not src.watched:
                return  # silently skip unwatched source

            # --- three-tier dedup (cheapest first) ---

            # Tier 0: same (user, channel, message) instance ? already processed.
            existing = await db.get_video(conn, user.id, channel_id, message_id)
            if existing is not None:
                return

            # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
            if document_id is not None:
                existing = await db.get_video_by_document_id(conn, user.id, document_id)
                if existing is not None:
                    log.info(
                        "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
                        "original_channel=%s original_msg=%s",
                        user.id,
                        document_id,
                        channel_id,
                        message_id,
                        existing.channel_id,
                        existing.message_id,
                    )
                    return

            # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
            if (
                meta.size_bytes is not None
                and meta.duration_s is not None
```

4. user (2026-07-06T09:09:20.801Z)

```
# ADR 0009: Selective channel scanning and multiplexed routing

- **Status**: Accepted
- **Date**: 2026-07-04
- **Decision owner**: project owner

## Context

Today the userbot scans **every** channel/group the account is a member of.
`manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
filter, and `on_new_message` processes every video regardless of source. The
`channels` table exists but is populated lazily as a *record* of seen channels;
it is not used as a filter.

The product vision is a **multiplexing video re-distribution system**:

- A small set of high-quality **source** channels (e.g. spiritual, classical
  music) are scanned for incoming videos.
- Matches are routed to themed **destination** broadcast channels (e.g. a
  Reggae channel, a Rap channel) that subscribers opt into.
- A user therefore subscribes to *outputs* by topic, not to *inputs* by source.

```

```
This requires two capabilities the current architecture does not have:

1. **Selective scanning** ? choose which source channels are watched, rather
   than ingesting all of them.
2. **Per-source ? per-destination routing** ? a match from source channel X is
   delivered to a specific destination channel Y, instead of the single global
   `delivery_target` (ADR 0005).

## Decision

Add three additive, independently-shippable capabilities:

### 1. Selective scanning (watched-flag)

Add a `watched` flag to the `channels` table. The default is locked as
**opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they
explicitly `/watch` a channel. This matches the multiplexing product shape
(sources are a *curated* set) and avoids blindly re-ingesting every chat the
account is a member of.

Gate `on_new_message` at the top: after computing `channel_id`, look up the
row and `return` early if `watched` is false, before any dedup or filter work.

This is a small migration plus ~5 lines in `telethon_client.py`.

**Discoverability note.** With `DEFAULT 0` plus the current lazy population
(a `channels` row is only inserted when a message is first seen), opt-in is
necessary but not sufficient for discoverability: a channel the account has
never received a message from has no row to flag. The `/channels` command
(step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
so users can browse and watch channels they have not yet received traffic
from. The flag default and the enumeration flow are orthogonal ? both are
required.

### 2. Channel enumeration and management commands

New Saved Messages commands (and, via ADR 0008, bot commands) building on the
existing `commands.py` pattern:

- `/channels` ? list the user's channels (via `client.get_dialogs()`, already
  used in `backfill.py`) with their watched state.
- `/watch <#name|id>` ? set `watched=1` for a source channel.
- `/unwatch <#name|id>` ? set `watched=0`.
- `/watchlist` ? list only watched sources.

### 3. Per-source ? per-destination routing rules

Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
refers to the same concept and should be aligned to `routing_rules`), e.g.:

```
routing_rules (
  user_id            INTEGER NOT NULL REFERENCES users(id),
  source_channel_id  INTEGER NOT NULL,
  dest_channel_id    INTEGER NOT NULL,   -- a broadcast channel / @username
  filter_config_id   INTEGER,            -- optional per-route filter override
  PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
)
```

On a match, look up applicable routing rules by `source_channel_id` and
deliver to each matching destination. The delivery layer (`delivery.py`)
already accepts a `target` per call, so this generalises the existing single
`delivery_target` to a per-match lookup. Users with no routing rules fall
back to the current global `delivery_target` behaviour (ADR 0005), preserving
```

backwards compatibility.

Consequences

Positive
- Enables the multiplexing product shape: scan curated sources, publish to topical broadcast channels, let subscribers pick outputs by interest.
- Each capability is additive and independently deployable; none disturbs the existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
- Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original* file, so a routed re-upload is not re-processed.
- Channel enumeration reuses `client.get_dialogs()` already present in `backfill.py`; no new Telethon capability is required.

Negative
- New schema (`channels.watched`, `routing_rules`) with migrations.
- **Routing loop invariant.** With multiplexed routing, the userbot account may be a member of a *destination* broadcast channel and re-see its own output, producing an infinite forward loop. The contract is: **a channel is either a source or a destination, never both.** This is enforced in `on_new_message` by excluding any channel that is a configured destination (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE user_id = ?)`) before the watched-flag check. Issue #25 Track E implements this; the invariant is stated here so the handler has a contract to code against.
- Per-route filter overrides add config complexity; v1 may ship with global filters only and a single implicit route (`source ? global target`), then layer explicit per-route rules later.

Backwards compatibility

Adding `channels.watched` with `DEFAULT 0` would silently stop every existing install from receiving matches until the user re-watches their channels ? a behaviour change disguised as a schema change. To preserve the current "ingest everything" semantics for existing installs, the migration backfills:

```sql
ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
```

i.e. any channel the install has *already* seen traffic from is implicitly watched, so pre-migration users observe no change. New channels discovered after the migration default to `watched = 0` and must be explicitly watched ? which is the intended new UX. This mirrors the "old rows behave exactly as before" pattern established by ADR 0005.

Rollout order

1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`, `/channels`). Smallest unit; immediately useful even without routing.
2. **Named destination channels** ? extend `/set delivery_target` to allow switching among a user-defined set, not just one global value.
3. **Per-source ? per-destination routing rules** (`routing_rules` table + `/route <source> <dest>` command). The full multiplexing vision.

Each step is shippable and useful on its own; step 3 is not required to deliver value from steps 1?2.

Alternatives considered

- **Server-side `chats=` filter at handler registration:** rejected ? the watched set is per-user and mutable at runtime; re-registering Telethon handlers on every `/watch` change is brittle. An in-handler lookup against the `channels` table is simpler and hot-swappable.

```
153     - **Content-based classification (auto-detect genre):** explicitly out of
154       scope for v1. Routing is user-defined by source channel; automatic topic
155       detection could be a future enhancement but is not required to deliver the
156       multiplexing UX.
157
158     ## References
159
160     - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161     - ADR 0005 (per-user delivery target) ? the single-target delivery this
162       generalises into per-route targets.
163     - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164       the original `document_id`.
165     - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166       `/unwatch`, and routing commands to public users.
167
```

5. assistant (2026-07-06T09:09:23.208Z)

I've read the gate and the ADR. Now let me verify the coverage claim by grepping the tests directory.

6. user (2026-07-06T09:09:24.825Z)

No matches found

7. user (2026-07-06T09:09:25.639Z)

Found 2 files
tests\test_command_service.py
tests\test_db.py

8. assistant (2026-07-06T09:09:29.292Z)

Let me check what those two files test regarding watched, and confirm no test drives the ingestion decision.

9. user (2026-07-06T09:09:30.859Z)

```
247:    watched: bool = False,
257:            watched=watched,
265:    await _seed_source(conn, USER_ID, "-1001", "Channel A", watched=True)
266:    await _seed_source(conn, USER_ID, "-1002", "Channel B", watched=False)
270:    assert "?" in result.reply_text  # watched indicator
271:    assert "?" in result.reply_text  # unwatched indicator
279:async def test_watchlist_only_watched(conn: aiosqlite.Connection, svc: CommandService) ->
None:
280:    await _seed_source(conn, USER_ID, "-1001", "Ch A", watched=True)
281:    await _seed_source(conn, USER_ID, "-1002", "Ch B", watched=False)
289:    assert "No watched sources" in result.reply_text
293:    await _seed_source(conn, USER_ID, "-1001", "Target", watched=False)
298:    assert src.watched is True
302:    await _seed_source(conn, USER_ID, "@mychan", "My Channel", watched=False)
307:    assert src.watched is True
311:    await _seed_source(conn, USER_ID, "-1001", "Target", watched=True)
316:    assert src.watched is False
326:async def test_set_source_watched_toggle(conn: aiosqlite.Connection) -> None:
327:    src = await _seed_source(conn, USER_ID, "-2000", watched=True)
329:    await db.set_source_watched(conn, USER_ID, src.id, False)
332:    assert reloaded.watched is False
335:async def test_list_account_sources_watched_only(
338:    await _seed_source(conn, USER_ID, "-10", watched=True)
339:    await _seed_source(conn, USER_ID, "-20", watched=False)
342:    watched = await db.list_account_sources(conn, USER_ID, watched_only=True)
343:    assert len(watched) == 1
```

```
344:    assert watched[0].provider_source_id == "-10"
```

10. user (2026-07-06T09:09:31.982Z)

```
748:            "SELECT account_id, provider, provider_source_id, title, watched " "FROM
sources"
983:            watched=True,
987:    fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-100555")
990:    assert fetched.watched is True
995:    """Re-upserting the same (account, provider, source_id) updates title/watched."""
999:        Source(account_id=USER_ID, provider="telegram", provider_source_id="-1",
watched=True),
1008:            watched=False,
1012:    assert updated.watched is False
1014:    fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
1016:    assert fetched.watched is False
1203:        # sources backfilled with watched=1 (ADR 0009 backfill).
1205:            "SELECT provider_source_id, watched FROM sources ORDER BY provider_source_id"
1392:            "SELECT title, watched FROM sources WHERE provider_source_id = '-1002'"
1397:        assert orphan[1] == 0  # watched=0 (no channels row)
1399:        # Known channel (-1001) has title + watched=1 from the channels row.
1401:            "SELECT title, watched FROM sources WHERE provider_source_id = '-1001'"
```

11. assistant (2026-07-06T09:09:42.426Z)

The tests confirm the finding's factual claims. The tests exercise:
- Command layer (`/watch`, `/unwatch`, `/channels`, `/watchlist`)
- DB layer (`set_source_watched`, `list_account_sources`, `get_source_by_provider_id`, backfill migration)

But nothing drives `on_new_message` or the gate logic (lines 184-191). The gate itself is inside `# pragma: no cover` (line 167). This all checks out.

Now let me adversarially assess the finding. The finding is categorized as `tests_ux` and claims severity `high`. Let me examine the reasoning carefully.

The finding's core claim: the gate has zero test coverage because it's inlined in a `# pragma: no cover` handler. This is factually true. However, the severity argument rests heavily on "the auto-watch defect in finding #1 shipped undetected." Let me examine what the gate actually does regarding the auto-watch behavior, to judge whether the missing coverage is genuinely high-severity.

Looking at lines 189-191:
```python
src = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
if src is not None and not src.watched:
    return  # silently skip unwatched source
```

This means: if `src is None` (a previously-unseen channel with no source row), the gate does NOT skip ? it falls through to the legacy path that upserts with `watched=True`. Per ADR 0009 line 130-131, new channels should default to `watched=0` and NOT be ingested. So the gate does have a real behavioral gap for unseen channels. Let me verify the legacy upsert path to confirm.

12. user (2026-07-06T09:09:43.377Z)

```
218                 meta.size_bytes is not None
219                 and meta.duration_s is not None
220                 and meta.height is not None
221                 and meta.width is not None
222             ):
223                 existing = await db.get_video_by_content_fingerprint(
224                     conn,
225                     user.id,
```

```
226                     meta.size_bytes,
227                     meta.duration_s,
228                     meta.height,
229                     meta.width,
230                 )
231                 if existing is not None:
232                     log.info(
233                         "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
234                         "channel=%s msg=%s original_channel=%s original_msg=%s",
235                         user.id,
236                         meta.size_bytes,
237                         meta.duration_s,
238                         meta.height,
239                         meta.width,
240                         channel_id,
241                         message_id,
242                         existing.channel_id,
243                         existing.message_id,
244                     )
245                     return
246 
247             # Lazily record the channel (best-effort; failures are non-fatal).
248             try:
249                 await db.upsert_channel(
250                     conn,
251                     Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
252                 )
253             except Exception:
254                 log.warning(
255                     "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
256                 )
257 
258             result = engine_holder[ENGINE_KEY].evaluate(meta)
259             record = VideoRecord(
260                 user_id=user.id,
261                 channel_id=channel_id,
262                 message_id=message_id,
263                 document_id=document_id,
264                 duration_s=meta.duration_s,
265                 width=meta.width,
266                 height=meta.height,
267                 size_bytes=meta.size_bytes,
268                 mime_type=meta.mime_type,
269                 matched=result.matched,
270             )
271             try:
272                 inserted = await db.insert_video(conn, record)
273             except sqlite3.IntegrityError:
274                 # Lost a dedup race: a concurrent delivery of the same file
275                 # (same document_id or same message instance) won the insert.
276                 # The three-tier check above already classified this as new, so
277                 # the collision means another task is processing (or has
```

13. assistant (2026-07-06T09:09:58.085Z)

Confirmed: for a previously-unseen channel (`src is None`), the gate does NOT return early. It falls through to dedup, then `upsert_channel` (line 249), which the prompt notes creates the source with `watched=True`, then `insert_video`. So an unseen channel's first video IS ingested and delivered ? a real ADR 0009 violation (lines 130-131). That's the referenced "finding #1" and it's genuine.

Now, this finding (tests_ux) is specifically about the *missing test coverage*, not the auto-watch bug itself. Let me evaluate its severity honestly.

The factual claims are all CONFIRMED:
- Gate is inside `# pragma: no cover` (line 167) ? true.
- No test references the gate/handler ? true (greps returned nothing for the handler).
- Coverage figure (97.10%) excludes the gate ? true by construction of the pragma.
- A test asserting "unseen channel's first video is NOT delivered" would have caught finding #1 ? true, since the gate as written does deliver it.

The one thing to scrutinize adversarially: is "zero test coverage on a `# pragma: no cover` handler" itself a *defect in this PR's diff*, or a pre-existing condition? The `on_new_message` pragma predates this PR (the whole handler is legacy). However, this PR *added new load-bearing logic* (the watched gate, lines 184-191) into that untested region without extracting it into anything testable. The gate is the entire point of the PR, and it shipped with a latent correctness bug that a single unit test would have caught. So the missing coverage is attributable to a choice made *in this diff*.

On severity: the finding claims `high`. The missing-coverage issue itself is a testing-hygiene gap, not a user-facing bug ? the underlying user-facing bug is finding #1. A test gap that lets a shipped bug through is real, but "high" overstates it as an independent finding; the actual user-facing damage is already counted in finding #1. As a standalone tests/hygiene finding, medium is the honest severity: it's a genuine, correctly-diagnosed coverage gap on the PR's core logic (worse than pure hygiene because it directly enabled an escaped bug), but it is not itself a broken-feature/data-loss issue.

14. user (2026-07-06T09:10:06.092Z)

Structured output provided successfully